

LAPORAN TUGAS 3

SISTEM OPERASI



**Dosen Pengampu Mata Kuliah:
Triawan Adi Cahyanto, M.Kom**

**Disusun Oleh:
Mohamad Qowi Aizani (2410651003)**

**PRODI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
2025**

Tugas 1: Kalkulator Paralel

Buat file kalkulator_pararel.c

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
double a, b;
```

```
double hasil_tambah, hasil_kurang, hasil_kali, hasil_bagi;
```

```
// Fungsi untuk setiap operasi
```

```
void* tambah(void* arg) {
```

```
    hasil_tambah = a + b;
```

```
    pthread_exit(NULL);
```

```
}
```

```
void* kurang(void* arg) {
```

```
    hasil_kurang = a - b;
```

```
    pthread_exit(NULL);
```

```
}
```

```
void* kali(void* arg) {
```

```
    hasil_kali = a * b;
```

```
    pthread_exit(NULL);
```

```
}
```

```
void* bagi(void* arg) {
```

```
    if (b != 0)
```

```
        hasil_bagi = a / b;
    else
        hasil_bagi = 0; // Hindari pembagian oleh nol
    pthread_exit(NULL);
}
```

```
int main() {
    pthread_t t1, t2, t3, t4;

    // Input dua angka
    printf("Masukkan angka pertama: ");
    scanf("%lf", &a);
    printf("Masukkan angka kedua: ");
    scanf("%lf", &b);

    // Membuat thread untuk tiap operasi
    pthread_create(&t1, NULL, tambah, NULL);
    pthread_create(&t2, NULL, kurang, NULL);
    pthread_create(&t3, NULL, kali, NULL);
    pthread_create(&t4, NULL, bagi, NULL);

    // Tunggu semua thread selesai
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(t3, NULL);
    pthread_join(t4, NULL);
}
```

```

// Tampilkan hasil
printf("\n=== Hasil Perhitungan ===\n");
printf("Penjumlahan: %.2lf\n", hasil_tambah);
printf("Pengurangan: %.2lf\n", hasil_kurang);
printf("Perkalian : %.2lf\n", hasil_kali);
if (b != 0)
    printf("Pembagian : %.2lf\n", hasil_bagi);
else
    printf("Pembagian : Tidak terdefinisi (pembagian oleh nol)\n");

return 0;
}

```

Output:

```

qowi@QOWIAIZANI:~$ gcc -o kalkulator_paralel kalkulator_paralel.c -lpthread
qowi@QOWIAIZANI:~$ ./kalkulator_paralel
Masukkan angka pertama: 15
Masukkan angka kedua: 5

=== Hasil Perhitungan ===
Penjumlahan: 20.00
Pengurangan: 10.00
Perkalian   : 75.00
Pembagian   : 3.00
qowi@QOWIAIZANI:~$

```

Analisis:

- Output menampilkan hasil operasi dasar aritmetika.
- Hasil sudah dalam format floating point dengan dua desimal.
- Operasi penjumlahan, pengurangan, perkalian, dan pembagian berjalan dengan benar.

Tugas 2: File Processing

Buat file file_processing.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>
#include <time.h>

#define MAX_SIZE 1000000

char *file_content;
long file_size;

// Fungsi menghitung jumlah kata dalam potongan teks
int hitung_kata(char *text, long start, long end) {
    int count = 0;
    int in_word = 0;

    for (long i = start; i < end; i++) {
        if (text[i] == ' ' || text[i] == '\n' || text[i] == '\t') {
            in_word = 0;
        } else if (!in_word) {
            in_word = 1;
            count++;
        }
    }
    return count;
}

// Struktur untuk argumen thread
typedef struct {
```

```
    long start;
    long end;
    int result;
} ThreadData;
```

```
void* thread_count(void *arg) {
    ThreadData *data = (ThreadData*) arg;
    data->result = hitung_kata(file_content, data->start, data->end);
    pthread_exit(NULL);
}
```

```
// Fungsi membaca file ke memori
```

```
void baca_file(const char *filename) {
    FILE *f = fopen(filename, "r");
    if (!f) {
        printf("Gagal membuka file.\n");
        exit(1);
    }
```

```
    fseek(f, 0, SEEK_END);
    file_size = ftell(f);
    rewind(f);
```

```
    file_content = malloc(file_size + 1);
    fread(file_content, 1, file_size, f);
    file_content[file_size] = '\0';
    fclose(f);
}
```

```
int main() {
    char filename[100];
```

```

printf("Masukkan nama file (contoh: input.txt): ");
scanf("%s", filename);

baca_file(filename);

clock_t start, end;
double waktu_single, waktu_multi;

// ===== SINGLE THREAD
=====

start = clock();
int total_single = hitung_kata(file_content, 0, file_size);
end = clock();
waktu_single = ((double)(end - start)) / CLOCKS_PER_SEC;

// ===== MULTI THREAD
=====

pthread_t t1, t2;
ThreadData d1, d2;

long mid = file_size / 2;
d1.start = 0;
d1.end = mid;
d2.start = mid;
d2.end = file_size;

start = clock();
pthread_create(&t1, NULL, thread_count, &d1);
pthread_create(&t2, NULL, thread_count, &d2);
pthread_join(t1, NULL);
pthread_join(t2, NULL);

```

```

int total_multi = d1.result + d2.result;

end = clock();

waktu_multi = ((double)(end - start)) / CLOCKS_PER_SEC;

// ===== HASIL =====

printf("\n=== HASIL PERHITUNGAN ===\n");
printf("Jumlah kata (Single Thread): %d\n", total_single);
printf("Jumlah kata (Multi Thread ) : %d\n", total_multi);
printf("\n=== WAKTU EKSEKUSI ===\n");
printf("Single-thread : %.6f detik\n", waktu_single);
printf("Multi-thread : %.6f detik\n", waktu_multi);

free(file_content);

return 0;

}

```

Output:

```

qowi@QOWIAIZANI:~$ gcc -o file_processing file_processing.c -lpthread
qowi@QOWIAIZANI:~$ ./file_processing
Masukkan nama file (contoh: input.txt): input.txt

=== HASIL PERHITUNGAN ===
Jumlah kata (Single Thread): 17
Jumlah kata (Multi Thread ) : 18

=== WAKTU EKSEKUSI ===
Single-thread : 0.000003 detik
Multi-thread : 0.000378 detik
qowi@QOWIAIZANI:~$ \

```

Analisis:

- Hasil Perhitungan Kata:
 - Single Thread: 17 kata
 - Multi Thread: 18 kata

Ada perbedaan 1 kata antara single-thread dan multi-thread, yang mungkin disebabkan oleh perbedaan cara pembagian tugas atau sinkronisasi data antar thread.

- Waktu Eksekusi:
 - Single-thread: 0.000003 detik (sangat cepat)
 - Multi-thread: 0.000378 detik (jauh lebih lama)

Ini menunjukkan bahwa untuk masalah ini, penggunaan multi-thread justru membuat eksekusi lebih lambat, kemungkinan karena overhead pembuatan dan sinkronisasi thread lebih besar daripada manfaat pembagian tugas.

Tugas 3: Producer-Consumer

Buat file producer_consumer.c

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define BUFFER_SIZE 10

int buffer[BUFFER_SIZE];
int count = 0; // jumlah item dalam buffer
int in = 0;    // index penulisan
int out = 0;   // index pembacaan

pthread_mutex_t mutex;
pthread_cond_t not_full;
pthread_cond_t not_empty;

// Fungsi Producer
void* producer(void* arg) {
    while (1) {
        int item = rand() % 100; // angka random
```

```

pthread_mutex_lock(&mutex);

// Tunggu jika buffer penuh
while (count == BUFFER_SIZE)
    pthread_cond_wait(&not_full, &mutex);

// Masukkan item ke buffer
buffer[in] = item;
in = (in + 1) % BUFFER_SIZE;
count++;

printf("Producer menghasilkan: %d | Jumlah item: %d\n", item,
count);

// Beri sinyal ke consumer
pthread_cond_signal(&not_empty);
pthread_mutex_unlock(&mutex);

sleep(1); // jeda agar terlihat prosesnya
}
}

// Fungsi Consumer
void* consumer(void* arg) {
    while (1) {
        pthread_mutex_lock(&mutex);

        // Tunggu jika buffer kosong
        while (count == 0)
            pthread_cond_wait(&not_empty, &mutex);
    }
}

```

```

        // Ambil item dari buffer
        int item = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
        count--;

        printf("Consumer memproses : %d | Sisa item: %d\n", item, count);

        // Beri sinyal ke producer
        pthread_cond_signal(&not_full);
        pthread_mutex_unlock(&mutex);

        sleep(2); // jeda agar terlihat prosesnya
    }
}

int main() {
    pthread_t prod, cons;

    // Inisialisasi mutex dan condition variable
    pthread_mutex_init(&mutex, NULL);
    pthread_cond_init(&not_full, NULL);
    pthread_cond_init(&not_empty, NULL);

    // Buat thread producer dan consumer
    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    // Tunggu thread (tidak akan selesai kecuali dihentikan manual)
    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

```

```

// Hancurkan mutex dan cond
pthread_mutex_destroy(&mutex);
pthread_cond_destroy(&not_full);
pthread_cond_destroy(&not_empty);

return 0;
}

```

Output:

```

qowi@QOWIAIZANI:~$ gcc -o producer_consumer producer_consumer.c -lpthread
qowi@QOWIAIZANI:~$ ./producer_consumer
Producer menghasilkan: 83 | Jumlah item: 1
Consumer memproses : 83 | Sisa item: 0
Producer menghasilkan: 86 | Jumlah item: 1
Consumer memproses : 86 | Sisa item: 0
Producer menghasilkan: 77 | Jumlah item: 1
Producer menghasilkan: 15 | Jumlah item: 2
Consumer memproses : 77 | Sisa item: 1
Producer menghasilkan: 93 | Jumlah item: 2
Producer menghasilkan: 35 | Jumlah item: 3
^C
qowi@QOWIAIZANI:~$ |

```

Analisis:

- Program berhasil menerapkan konsep producer-consumer.
- Producer berhasil memasukkan item ke dalam buffer dengan jumlah yang bertambah.
- Consumer berhasil mengambil dan memproses item tersebut, menyebabkan jumlah item berkurang.
- Sinkronisasi antar thread berjalan cukup baik terlihat dari output, meski tidak pasti tanpa melihat kodenya.
- Output juga menunjukkan bahwa buffer bisa menampung lebih dari satu item (Jumlah item sampai 3).